PSID Crisis Module Client Series Technical Deliverable

Client 1: Technical Codebook

Refreshed PSID crisis-module workflow, scoring logic, and artifact schema

Prepared for client review | Updated: April 2026 | https://github.com/namo507/Team-PSID

Abstract

This technical codebook documents the refreshed PSID crisis-module workflow from the ranked source bank to the scored CSV, figures, spreadsheet, and client PDFs. It explains the scoring formulas, the 28-row scored selection, the 26-question deployable instrument, and the optimization choices that now govern portability, redundancy control, and respondent-aware routing.

Keywords: PSID crisis module | utility-burden optimization | technical codebook | deployable survey design

---

1 Purpose and Scope

This technical codebook documents the refreshed PSID crisis-module workflow from the authoritative ranked source file through the final scored CSV, figures, spreadsheet, and PDF bundle. It is written to support analytic review, reproducibility, and downstream client sign-off.

Production status

- 52 ranked historical questions in the current corpus.
- 28 selected rows in the scored module and 26 questions in the deployable instrument.
- 3 `Generic` rows and 49 `Specific` rows after binary thresholding.
- 29.98 selected minutes in the scored module and 28.82 minutes in the deployable instrument.
- The 28-to-26 change comes from excluding `SPC_AGE_YEARS` and `SPC_DESCRIBE_SEX_GENDER` from routine field administration while retaining them for traceability.

2 Headline Benchmark Summary

Metric	Value	How the number is used
Total ranked questions	52	Count of all rows in the integrated source bank; defines the full analytic universe.
Selected scored questions	28	Count of shortlisted rows under the 30-minute review budget.
Deployable questions	26	Scored shortlist minus the two selected demographic-only traceability rows.
Selected scored minutes	29.98	Summed minutes across the 28 scored rows; confirms the review package stays inside the budget.
Deployable minutes	28.82	Summed minutes across the field instrument after age and sex/gender are removed from default administration.
Full corpus minutes	68.60	Summed minutes across all 52 rows; shows why selection is needed.
Generic rows	3	Count of rows above the normalized threshold; defines the portable universal core.
Specific rows	49	Remaining rows in the context-sensitive bank.
Average R_i	2.201	Average baseline signal-per-burden score across the refreshed corpus.
Maximum R_i	5.667	Top baseline efficiency score and normalization ceiling in the current refresh.
Average P_i	2.561	Average distinctiveness-adjusted priority score across the refreshed corpus.
Maximum P_i	7.860	Top augmented priority score after portability and redundancy adjustments.

Table 1: Current benchmark summary used by the maintained client package.

3 Why 28 Scored Rows Become 26 Deployable Questions

Stage	Rows	Minutes	Calculation / rule	Why it is retained or removed
Full ranked corpus	52	68.60	Count of all rows in the integrated source; minutes are summed across the corpus	Starting point for ranking, diagnostics, and documentation.
Scored module	28	29.98	Count of selected rows after ranking and time-budget filtering	Preserves the full analytic shortlist, including traceability fields.
Selected demographic traceability rows	2	1.17	`SPC_AGE_YEARS` (0.35) + `SPC_DESCRIBE_SEX_GENDER` (0.817)	Useful for framing and optional routing, but comparatively weak as direct crisis-impact questions.
Deployable instrument	26	28.82	$28 - 2 = 26$ and $29.98 - 1.167 = 28.813 \approx 28.82$	Final asked instrument prioritizes direct crisis-impact content instead of forcing demographic context into every interview.

Table 2: The change from 28 scored rows to 26 deployable questions is a deployment optimization, not a re-ranking of the corpus.

Interpretation

The model still selects 28 rows as the best analytic bundle under the budget. The deployable version removes only the two demographic-only rows because they are better treated as traceability or conditional-routing variables than as routine crisis-module questions.

4 Figures and Visual Diagnostics

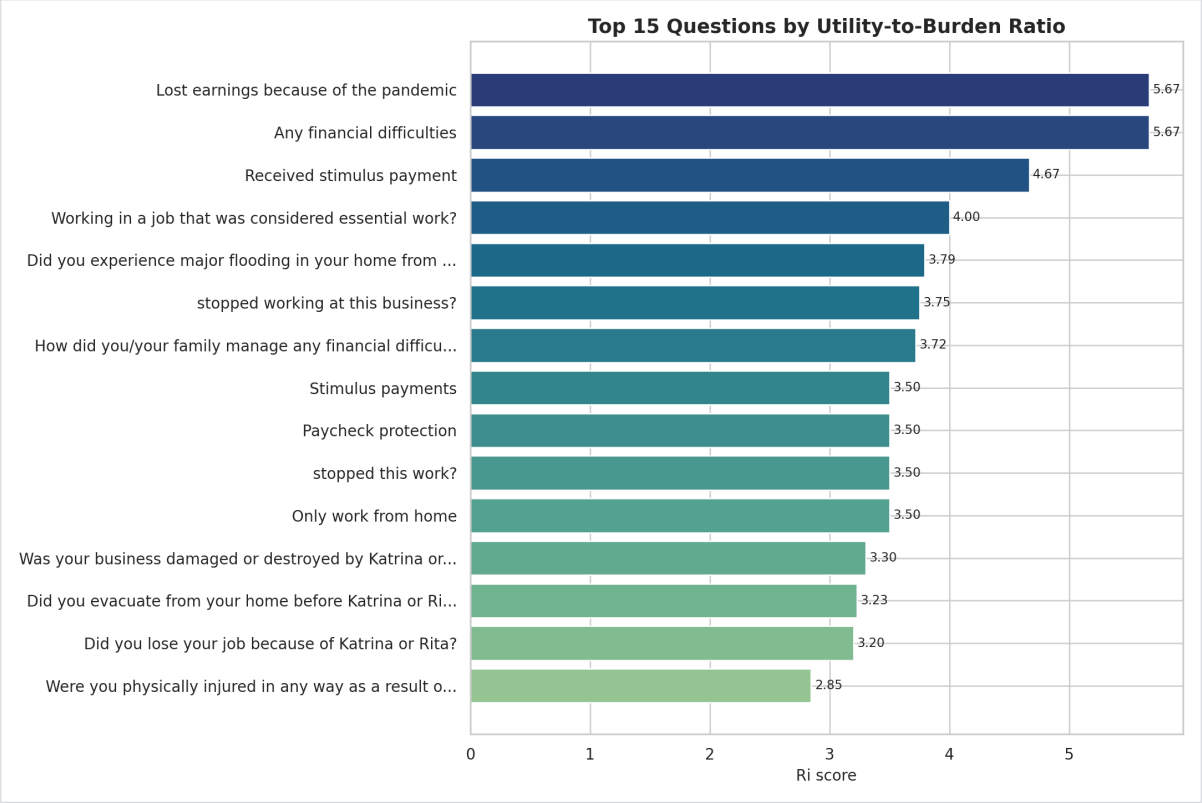

Figure 1: Top-ranked questions from the refreshed corpus.

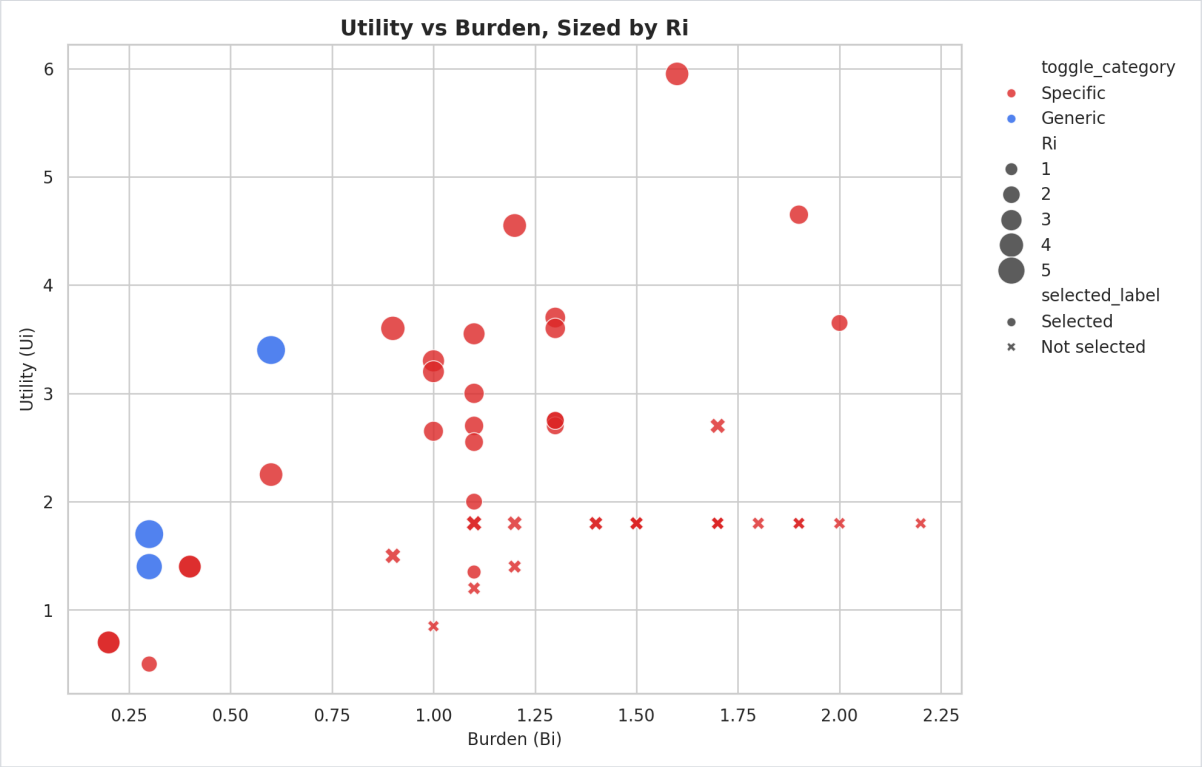

Figure 2: Utility-versus-burden view used to inspect signal efficiency.

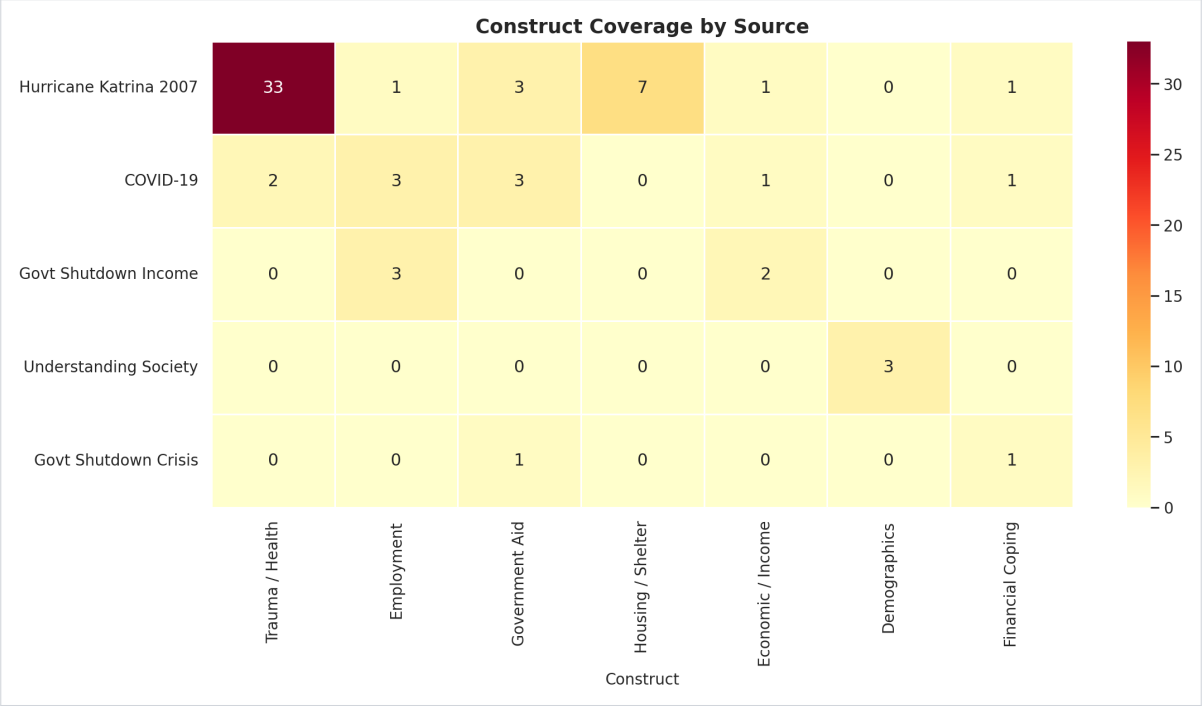

Figure 3: Construct coverage heatmap for the refreshed item bank.

5 End-to-End Workflow Pipeline

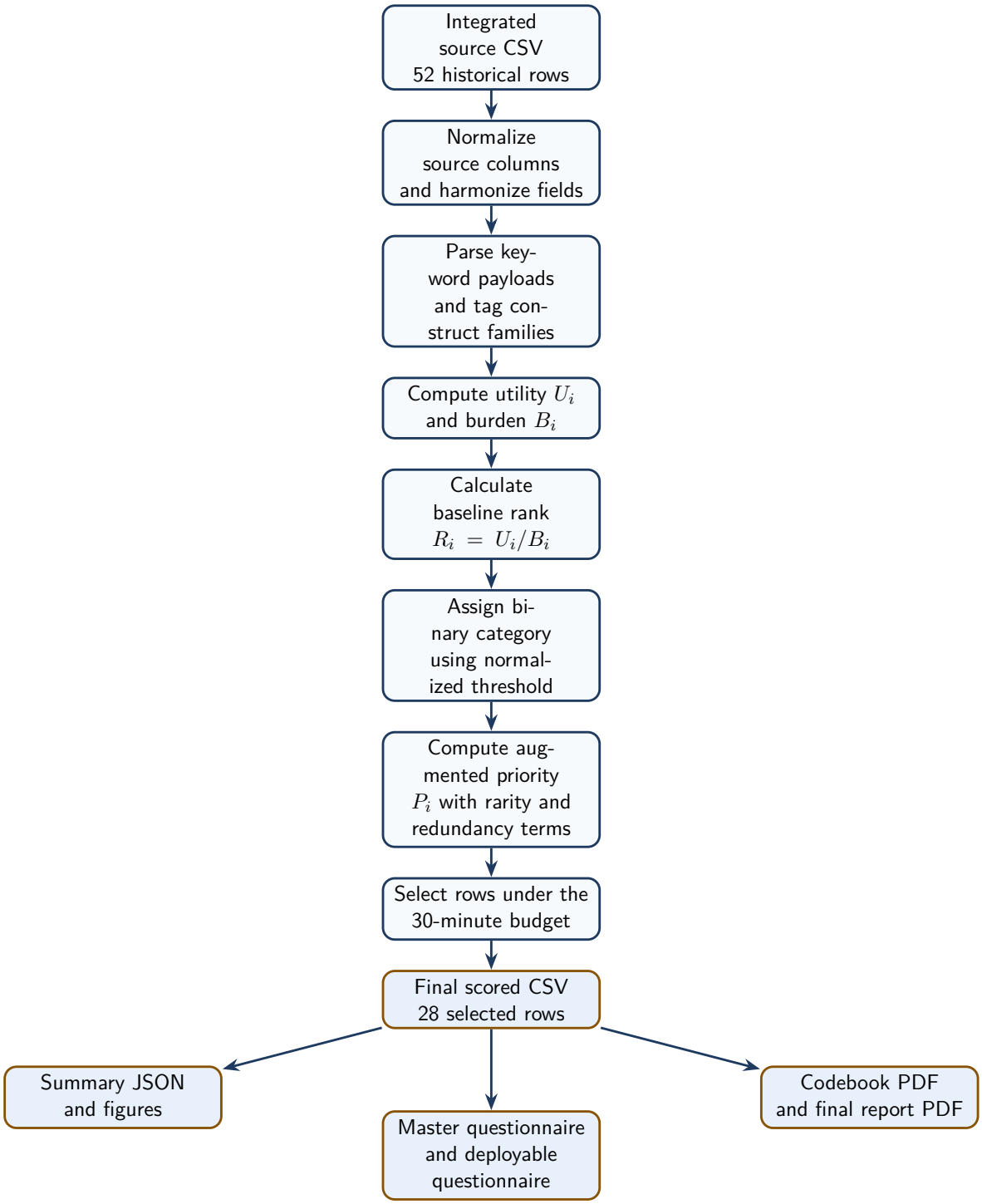

Figure 4: Production workflow from the raw integrated source to the maintained client package.

6 Optimization Strategy Change

Dimension	Legacy architecture view	Current production view and implication
Client-facing categories	Legacy architecture exposed 7 Generic Core rows, 1 Financial Crisis row, and 20 Pandemic / Disaster rows inside the 28-row shortlist.	Current production uses 3 Generic and 25 Specific rows in the scored module, with 3 Generic and 23 crisis-specific rows in the deployable instrument. The binary story is simpler and the universal core is narrower.
Financial representation	Finance appeared as its own visible toggle label.	Financial rows now compete inside Specific and are moderated by redundancy penalties, so near-duplicate earnings and aid items no longer create a separate finance-only bank.
Deployment target	The scored shortlist was closer to the field-facing view.	The 28-row scored file is retained for auditability, while the 26-row deployable file is used for routine administration. This separates analytic traceability from operational burden.
Optimization levers	The older view emphasized score plus legacy toggle structure.	The current workflow adds portability, construct coverage, redundancy control, and documentation-backed wording replacements to support an always-on, cross-crisis instrument.
Routing refinement	Category labels described broad source-specific routing.	Generic answers and crisis identification now activate one relevant specific block, so respondent answers reduce effective burden without overwriting stored row scores.

Table 3: From the legacy multi-label architecture to the current production optimization strategy.

7 Respondent-Answer Refinement and Optimization

Refinement input	Quantitative effect	Why it matters and the assumption behind it
Universal Generic answers and crisis identification	Effective live path becomes the 3-item Generic core plus one specific block; pandemic path is $1.40 + 2.92 = 4.32$ instead of the full 26-question paper instrument.	This cuts unnecessary burden and keeps the interview aligned to the active crisis context. It assumes one dominant crisis context is active in a given administration.
Selected demographic answers	Removing `SPC_AGE_YEARS` (0.35) and `SPC_DESCRIBE_SEX_GENDER` (0.817) lowers the base deployable length by 1.167 minutes.	This preserves traceability without forcing low-yield context items into every interview. It assumes demographic information can be collected elsewhere or only when routing requires it.
Historical respondent distributions	Shutdown coping, stimulus uptake, and pandemic response patterns keep Government Aid and Financial Coping constructs salient in the selected bank.	This grounds construct weights and wording choices in observed crisis behavior rather than only lexical similarity.
Feed-forward respondent state	Prior-wave status, identity, address, and eligibility can refine universes or wording windows without overwriting stored U_i, B_i, R_i, or P_i values.	This separates static ranking from live survey routing and assumes prior-wave metadata is available when the module is deployed longitudinally.

Table 4: Respondent data influences deployment and construct emphasis even when the stored row scores remain fixed.

The respondent-data evidence used for refinement is substantive rather than cosmetic. In the reference evidence, 70.6% of shutdown-affected families cut spending, 28.3% used food banks, and pandemic response data show strong heterogeneity in vaccination and stimulus behavior. Those patterns explain why coping, aid, earnings-loss, and health-response items remain prominent after redundant variants are pruned.

8 Files and Roles

File	Role	Why it matters
`data/PSID_Ranked_Questions_Katrina_Integrated.csv`	Raw integrated source bank	Starting point for every production rebuild.
`PSID_NLP_Crisis_Module_Structure.py`	Shared scoring helpers	Holds constants, keyword parsing, burden logic, threshold helpers, and selection utilities.
`generate_psid_artifacts.py`	Production rebuild engine	Writes the final scored CSV, summary JSON, dashboard payload, and figures.
`data/PSID_Ranked_Questions_Final.csv`	Authoritative scored output	Main ranked table used by the notebook and client deliverables.
`PSID_NLP_Crisis_Module_Final.ipynb`	Validation notebook	Recomputes the workflow and verifies the persisted artifact bundle.
`generate_client_documents.py`	Client-package renderer	Builds the spreadsheet and PDFs from the scored CSV and summary JSON.
`data/psid_artifact_summary.json`	Metric bundle	Stores benchmark counts and score summaries for the latest run.

Table 5: Files that drive the maintained workflow and client package.

9 Core Scoring Logic

9.1 Utility, burden, and baseline rank

The baseline score is driven by question utility relative to response burden.

Core formulae

$$U_i = \sum \text{keyword weights}$$
$$B_i = \max(0.10 \times \text{word\_count} + 0.20 \times \text{complexity}, 0.1)$$
$$R_i = \frac{U_i}{B_i}$$

- U_i rewards crisis-relevant content signaled by tagged keywords.
- B_i penalizes long or complex wording.
- R_i ranks items that carry more signal per unit of burden.

9.2 Binary category assignment

The client requirement was a binary rule based on R_i, but the raw score is not naturally bounded in $[0, 1]$. The production pipeline therefore applies the cutoff to a min-max normalized score.

Operational threshold rule

$$\text{ri\_threshold\_score} = \frac{R_i - \min(R_i)}{\max(R_i) - \min(R_i)}$$

$$\text{Generic} \iff \text{ri\_threshold\_score} \geq 0.70$$

$$\text{Specific} \iff \text{ri\_threshold\_score} < 0.70$$

9.3 Augmented priority score

The final prioritization score preserves R_i while rewarding breadth and distinctiveness.

Priority adjustment

$$\text{augmented\_utility} = U_i \times \Big(1 + 0.32 \times \text{idf\_scaled} + 0.24 \times \text{construct\_scaled} + 0.12 \times \text{richness\_scaled} + \text{portability\_bonus}\Big)$$

$$P_i = \frac{\text{augmented\_utility}}{B_i \times (1 + 0.65 \times \text{redundancy\_scaled})}$$

- P_i helps separate distinct, portable items from repetitive variants.
- The shortlist is still bounded by the time budget, not only by raw rank.

9.4 Why each quantitative field exists

Quantity	Rule	Why it matters
U_i	Sum of matched keyword weights.	Captures crisis relevance; it assumes tagged keywords are a defensible proxy for item content.
B_i	Word-count and complexity penalty with a floor at 0.1.	Encodes respondent burden; it assumes burden grows with length and cognitive complexity.
R_i	U_i/B_i.	Measures signal per unit of burden, so higher values indicate more efficient questions.
`ri_threshold_score`	Min-max normalization of R_i.	Makes the client's 0.70 threshold meaningful on an otherwise unbounded score.
Augmented utility	U_i adjusted by rarity, construct breadth, richness, and portability.	Rewards distinct, portable coverage before redundancy is applied.
P_i	Augmented utility divided by burden and redundancy penalty.	Breaks ties among similar R_i values by down-weighting repetitive items.
minutes	(word_count $\times$ 7)/60.	Operational time field used in the budget and instrument assembly.
Selected rows	Time-budget filter after ranking.	Defines the scored module for review and traceability.
Deployable rows	Selected rows minus demographic-only traceability rows.	Defines the field instrument used in routine administration.

Table 6: Formula, purpose, and assumption guide for the main quantitative fields.

10 Worked Calculation Example

Example question: `GEN_LOSE_EARNINGS_PANDEMIC`
Recommended wording: *Did you lose earnings because of the pandemic?*

Field	Value	Calculation	Why it is used / assumption
U_i	3.400	$0.80 + 0.80 + 0.90 + 0.90$	Utility aggregates the matched crisis-signal weights; assumes the tagged keywords capture the substantive content of the item.
word count	6	Counted from deployable wording	Used in both burden and time estimates; assumes the cleaned wording is the administered string.
complexity	0.0	Precomputed complexity field	Prevents an additional burden penalty; assumes the wording is direct and does not require extra respondent processing.
B_i	0.600	$\max(0.10 \times 6 + 0.20 \times 0.0, 0.1)$	Converts wording cost into a penalty term; assumes the burden constants remain fixed design choices.
R_i	5.667	3.40/0.60	Measures signal efficiency; assumes high-value universal questions should deliver strong utility per unit of burden.
threshold score	1.000	$(5.667 - 0.818)/(5.667 - 0.818)$	Supports the binary split; assumes the current refresh min and max are the correct normalization anchors.
Final category	Generic	$1.000 \geq 0.70$	Places the item in the universal core; assumes the client wants only the top portable items to be Generic.
P_i	7.860	Value computed after rarity, construct, richness, portability, and redundancy scaling	Separates this item from similar crisis questions; assumes distinct portable items deserve priority in a short module.
minutes	0.70	$(6 \times 7)/60$	Feeds the time budget; assumes oral administration averages 7 seconds per word.

Table 7: Worked example for one of the highest-value Generic items.

11 Threshold Diagnostics

Threshold diagnostic	Value	Calculation / implication
Minimum raw R_i	0.818	$\min(R_i)$ across the 52-row corpus.
Maximum raw R_i	5.667	$\max(R_i)$ across the 52-row corpus.
Rows with raw $R_i \geq 0.70$	52 of 52	Shows that a direct raw cutoff would classify every row as Generic.
Rows with normalized threshold score ≥ 0.70	3 of 52	Final Generic count after min-max normalization.

Table 8: Why normalized thresholding is required for a meaningful binary split.

Interpretation

Raw $R_i \geq 0.70$ would classify every row as `Generic`. Normalized thresholding preserves the requested cutoff while still creating a real binary split between portable high-signal items and context-dependent items.

12 Construct Coverage and Interaction Notes

12.1 Selected construct coverage

Construct	Selected questions
Trauma / Health	15
Employment	7
Housing / Shelter	6
Government Aid	5
Economic / Income	4
Financial Coping	2
Demographics	2

Table 9: Construct coverage in the selected scored module.

12.2 Demographic interaction summary

Group	Rows	Average R_i	Average P_i
Demographics only	3	1.354	2.189
Non-demographic crisis items	49	2.253	2.584

Table 10: Demographic-only rows versus direct crisis-impact rows.

Interpretation

Demographic-only items remain useful for traceability and module framing, but they carry less direct crisis signal than employment, housing, aid, trauma, and economic disruption items. That is why the final deployable questionnaire excludes demographics even though the master review file retains them.

13 Output Schema

Field	Meaning
`variable_name`	Stable identifier used across the CSV, spreadsheet, and PDFs.
`question_text`	Historical source wording.
`recommended_wording`	Clean deployable wording.
`source`	Historical module source.
`toggle_category`	Final binary category: Generic or Specific.
`historical_toggle_category`	Legacy category kept for traceability.
`Ui`, `Bi`, `Ri`	Utility, burden, and baseline rank.
`ri_threshold_score`	Normalized rank used for binary categorization.
`Pi`	Enhanced priority score.
`selected_for_module`	Final selection flag under the time cap.
`minutes`	Estimated administration time.
`constructs`	Tagged construct families.

Table 11: Key fields preserved in the authoritative scored dataset.

14 Refresh Procedure

Use this order whenever the client package is refreshed:

1. Rebuild scored artifacts via `build_all()` in `generate_psid_artifacts.py`.

2. Validate notebook outputs in `PSID_NLP_Crisis_Module_Final.ipynb`.
3. Regenerate spreadsheet and PDFs through `generate_client_documents.py`.
4. Rebuild the client-facing LaTeX reports through `docs/latex/build_client_reports.sh`.
5. Confirm that metrics, figures, tables, and deliverable paths remain aligned.

That sequence keeps the scored CSV, the notebook, the figures, the packaged PDFs, and the presentation-ready report files synchronized.

A Appendix: Production Functions Used in the Workflow

A.1 Shared scoring and selection functions

Function	Script	Purpose
`normalize_ranked_questions()`	`PSID_NLP_Crisis_Module_Structure.py`	Renames source columns, parses keyword payloads, derives crisis-origin metadata, and normalizes legacy toggle labels before scoring.
`min_max_scale()`	`PSID_NLP_Crisis_Module_Structure.py`	Rescales a numeric series to the $[0, 1]$ interval for thresholding and other normalized comparisons.
`assign_binary_categories()`	`PSID_NLP_Crisis_Module_Structure.py`	Creates `ri_threshold_score` and applies the 0.70 cutoff that produces the final Generic versus Specific split.
`extract_keywords()`	`PSID_NLP_Crisis_Module_Structure.py`	Uses RAKE and noun-phrase extraction to generate candidate phrases from question text.
`tag_keywords()`	`PSID_NLP_Crisis_Module_Structure.py`	Maps extracted phrases onto the construct taxonomy and assigns the construct weights used in utility scoring.
`compute_word_count()`	`PSID_NLP_Crisis_Module_Structure.py`	Counts words for both burden and time-budget calculations.
`compute_complexity()`	`PSID_NLP_Crisis_Module_Structure.py`	Measures linguistic complexity from named entities and clause markers.
`compute_utility()`	`PSID_NLP_Crisis_Module_Structure.py`	Sums matched taxonomy weights into U_i.
`compute_burden()`	`PSID_NLP_Crisis_Module_Structure.py`	Converts word count and complexity into B_i.
`extract_constructs()`	`PSID_NLP_Crisis_Module_Structure.py`	Derives the unique construct set attached to each question.
`classify_toggle()`	`PSID_NLP_Crisis_Module_Structure.py`	Applies the rule-based legacy architecture labels: Generic Core, Financial Crisis, or Pandemic / Disaster.
`select_for_time_budget()`	`PSID_NLP_Crisis_Module_Structure.py`	Greedy selector that keeps the highest-ranked rows inside the 30-minute cap, prioritizing the Generic core first when present.

Table 12: Core preprocessing, scoring, and selection functions.

A.2 Artifact-generation and wording functions

Function	Script	Purpose
`suggest_deployable_wording()`	`generate_psid_artifacts.py`	Rewrites source-specific wording into cleaner deployable phrasing and applies documentation-backed replacements where exact wording is preferred.
`build_ranked_dataset()`	`generate_psid_artifacts.py`	Runs the end-to-end scoring pipeline, produces the derived fields, and returns the ranked dataset that feeds the downstream artifacts.
`compute_augmented_scores()`	`generate_psid_artifacts.py`	Computes TF-IDF rarity, redundancy, construct bonus, portability bonus, augmented utility, and the final P_i score.
`build_design_recommendations()`	`generate_psid_artifacts.py`	Converts scored questions into grouped design recommendations for the Generic and Specific banks.
`build_summary()`	`generate_psid_artifacts.py`	Aggregates headline counts, score summaries, source counts, and construct counts for dashboards and reports.
`write_dashboard_data()`	`generate_psid_artifacts.py`	Writes processed data and summary payloads for the dashboard and figure consumers.
`plot_top_ranked()`	`generate_psid_artifacts.py`	Generates the top-ranked-questions figure.
`plot_toggle_comparison()`	`generate_psid_artifacts.py`	Generates the category-comparison figure used to inspect the final distribution.
`plot_utility_vs_burden()`	`generate_psid_artifacts.py`	Builds the utility-versus-burden scatter plot sized by R_i.
`plot_construct_heatmap()`	`generate_psid_artifacts.py`	Produces the construct-coverage heatmap.
`plot_time_budget()`	`generate_psid_artifacts.py`	Produces the time-budget figure used in the deployable and stakeholder reports.

Table 13: Functions that generate the ranked dataset, wording, and figures.

A.3 Client-package assembly functions

Function	Script	Purpose
`build_questionnaire_frames()`	`generate_client_documents.py`	Creates the master and deployable questionnaire tables from the scored dataset, including Universes and rounded score fields.
`write_master_questionnaire()`	`generate_client_documents.py`	Writes and formats the internal traceability workbook.
`build_interaction_summary()`	`generate_client_documents.py`	Summarizes demographic and non-demographic interaction patterns for reporting.
`build_codebook_pdf()`	`generate_client_documents.py`	Renders the internal artifact-pipeline codebook PDF.
`build_report_pdf()`	`generate_client_documents.py`	Renders the internal artifact-pipeline stakeholder report PDF.
`build_deployable_pdf()`	`generate_client_documents.py`	Renders the internal artifact-pipeline deployable questionnaire PDF.

Table 14: Functions that package the scored dataset into internal review artifacts.

B Appendix: Detailed Formula Derivations and Value Explanations

B.1 Base scoring pipeline

For question i, the code builds the numeric fields in the following order:

$$
\begin{aligned}
\text{keyword\_list}_i &= \text{extract\_keywords}(\text{question\_text}_i) \\
\text{tagged\_keywords}_i &= \text{tag\_keywords}(\text{keyword\_list}_i) \\
U_i &= \sum_{k \in \text{tagged\_keywords}_i} w_k \\
\text{word\_count}_i &= \text{len}(\text{split}(\text{question\_text}_i)) \\
\text{complexity}_i &= \#\text{entities}_i + 0.5\Big(\#\text{commas}_i + \#\text{semicolons}_i + \#\text{ “or”}_i + \#\text{ “and”}_i\Big) \\
B_i &= \max\Big(0.10 \times \text{word\_count}_i + 0.20 \times \text{complexity}_i, 0.1\Big) \\
R_i &= \frac{U_i}{B_i}
\end{aligned}
$$

How to interpret these values

- U_i increases when more high-weight crisis constructs are matched.
- `word_count` is taken directly from the item text and feeds both burden and time estimates.
- `complexity` increases when a question contains more named entities or clause separators, which is a proxy for respondent processing burden.
- B_i has a floor of 0.1 so that very short questions never create an undefined or explosively large ratio.
- R_i is the baseline efficiency score: more crisis signal per unit of burden yields a larger value.

B.2 Binary thresholding, time-budget selection, and deployable minutes

The final client-facing category model uses a normalized version of R_i:

$$\text{ri\_threshold\_score}_i = \frac{R_i - \min(R)}{\max(R) - \min(R)}$$

$$\text{Generic} \iff \text{ri\_threshold\_score}_i \geq 0.70$$

$$\text{Specific} \iff \text{ri\_threshold\_score}_i < 0.70$$

The scored shortlist is then chosen by a greedy time-budget filter:

$$\text{question\_seconds}_i = \text{word\_count}_i \times 7$$

$$\text{selected\_for\_module}_i = 1 \text{ if adding row } i \text{ keeps cumulative time } \leq 1800 \text{ seconds}$$

The deployable instrument length is then derived from the scored shortlist, not from a second ranking pass:

$$\begin{aligned} \text{deployable minutes} &= 29.98 - 0.35 - 0.817 \\ &= 28.813 \approx 28.82 \end{aligned}$$

The subtraction removes `SPC_AGE_YEARS` and `SPC_DESCRIBE_SEX_GENDER` from routine field administration while leaving them available in the traceability record.

B.3 Augmented priority score and why values change after enrichment

The production workflow does not stop at R_i. It also computes a distinctiveness-adjusted priority score that favors portable and non-redundant items.

$$\begin{aligned}
\text{idf_strength}_i &= \text{mean(TF-IDF weights of analyzed tokens)} \\
\text{redundancy_penalty}_i &= \max_j \text{cosine_similarity}(i, j) \\
\text{rarity}_i &= \text{mean}\left(\frac{1}{\text{construct frequency}}\right) \times N \\
\text{priority}_i &= \text{mean(construct weights)} \\
\text{construct_bonus}_i &= \text{rarity}_i + \text{priority}_i + 0.08 \times \text{construct_count}_i
\end{aligned}$$

The portability bonus is a fixed rule based on whether a question is Generic or Specific and whether it contains source-specific terms such as *katrina*, *rita*, *fema*, *covid*, *pandemic*, or *shutdown*:

Row type	Source-specific term present?	Portability bonus
Generic row	No	0.16
Generic row	Yes	0.03
Specific row	No	0.08
Specific row	Yes	0.02

Table 15: Portability bonus rules used inside the augmented-priority calculation.

All scaled components use min-max scaling within the current refreshed dataset, and the final augmented score is:

$$\begin{aligned}
\text{augmented_utility}_i = U_i \times \Big(1 &+ 0.32\,\text{idf_scaled}_i + 0.24\,\text{construct_scaled}_i \\
&+ 0.12\,\text{richness_scaled}_i + \text{portability_bonus}_i\Big)
\end{aligned}$$

$$P_i = \frac{\text{augmented\_utility}_i}{B_i \times \left(1 + 0.65\,\text{redundancy\_scaled}_i\right)}$$

Why P_i can differ from R_i

Two questions can have similar baseline efficiency but very different P_i values. A question moves up when it adds rare or high-priority constructs, uses portable wording, and is not a near-duplicate of other shortlisted items. A question moves down when it is redundant, overly source-specific, or substantively repetitive even if its raw R_i is acceptable.